

Autoresearch Loops

A Causal Study of the Instruction File

Christian Block

Research project with Prof. Ramesh

September 7, 2026

An autoresearch loop runs its own experiments

- The model is given a goal, code it may edit, and an executor
- The executor runs one trial and returns a single scalar
- From there it proceeds without a human in the loop

The cycle

Propose a change, execute it, read the outcome, decide what to attempt next, repeat until the budget is exhausted.

- Progress is enforced by version control: `commit`, train, keep if the score improved, otherwise `git reset`

Why this matters now

- Karpathy's autoresearch, March 2026
- Forked several thousand times since
- It is what fixed the current meaning of the term
- People are running it against production systems. Famously Shopify CEO ->

Shopify Liquid

About 120 experiments

53% faster parse and render

Author's own verdict: probably overfitted

Never merged

The gap

The loops' functional behavior is unknown. And no prior work tries to do so. Such systems are evaluated end to end, on whether the pipeline produced an acceptable result, so the literature can say that a loop worked but never which part of its instruction file was responsible.

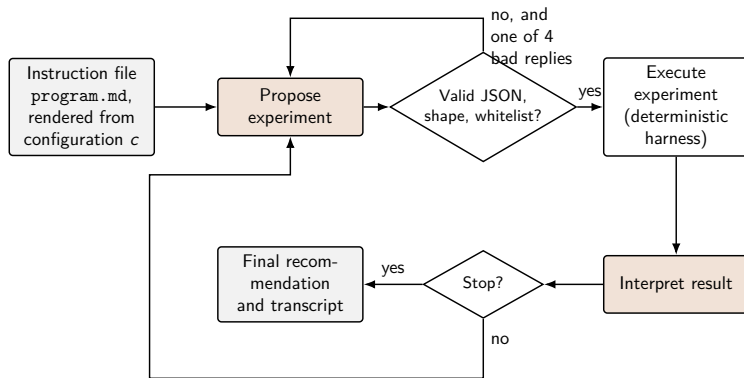
The architecture is three files, and one of them is the controller

File	Holds	Edited by
<code>prepare.py</code>	the data and the score	nobody
<code>train.py</code>	the model and the training loop	the agent
<code>program.md</code>	the instructions	the human

- `prepare.py` is unreachable, so every experiment is scored the same way
- The nine step experiment cycle exists only as English prose inside `program.md`, and the model itself is the execution layer

The observation the project rests on
`program.md` is the controller.

Figure 1: control flow of one run (loop/runner.py)



Shaded boxes are language model calls; the execute box is deterministic, so every number the loop reasons about was measured and not generated. The validity check is two checks in the code: the harness parses the reply and checks its shape, and `train.py` checks the whitelist before any estimator is fitted. Either way the proposal is logged as an attempt, which is what the wasted trial ratio counts, and costs a turn but not an experiment. A run ends on the model's stop decision, on 8 executed experiments, or on 4 bad replies.

Research questions

RQ1

Which components of `program.md` causally determine which aspects of the loop's research *process*, and how large is each effect?

RQ2

Do the components act independently of one another, or does the effect of one depend on the level of another?

- The contribution: The aim of what an existing autoresearch loop responds to

From prose to a configuration vector

- Every file is rendered from one template
- The fixed part gives the research question, the executor interface and the JSON response format, byte identical across all 32 files
- The variable part is five paragraphs, each written at one of two levels. Nothing else in the file varies

A file becomes a point in a space

$$c = (c_1, \dots, c_5), \quad c_i \in \{0, 1\} \quad \implies \quad 2^5 = 32 \text{ files}$$

The five components: the independent variables

	Component	Level 0	Level 1
<i>M</i>	Evaluation criterion	Success left undefined; the loop judges quality as it sees fit.	Success is 5 fold cross validated accuracy; the standard deviation breaks ties only.
<i>B</i>	Search breadth	Commit to one model family for the session.	Compare both families and vary several hyperparameters at once.
<i>S</i>	Stopping rule	Fixed budget of exactly three experiments.	Adaptive budget up to eight, halt when improvement falls below 0.002.
<i>O</i>	Reporting style	Terse: the action taken and the numbers involved.	Full sentences: hypothesis, expectation, interpretation.
<i>E</i>	Search emphasis	Explore first: several configurations before refining any.	Exploit first: refine a promising configuration immediately.

These categories were not invented for the study. A production `program.md` hardcodes a baseline to exceed, specifies failure handling, fixes a reporting discipline and imposes a parsimony rule.

Every file is rendered from this template

```
# Research Task: {{ task_title }}                                <- set by the task, not by c

You are an autonomous research agent. Your task is to investigate the following
question through real, executed experiments (not simulated ones):

> {{ research_question }}                                       <- set by the task, not by c

You have access to a fixed dataset ({{ dataset_name }}, already loaded and split
into train/validation) and to a tool that fits either a RandomForestClassifier or
a GradientBoostingClassifier with hyperparameters you choose. The tool always
returns the same record: 5-fold cross-validated accuracy on the training split,
its standard deviation across folds, and accuracy on the held-out validation split.

## Evaluation
{{ metric_text }}                                              <== slot M

## Search strategy
{{ breadth_text }}                                             <== slot B
{{ emphasis_text }}                                             <== slot E

## Budget and stopping
{{ stopping_text }}                                           <== slot S

## Reporting style
{{ output_format_text }}                                       <== slot 0

## Required response format
At every step, respond with a single JSON object and nothing else, matching one of these two shapes:
{"action": "propose", "hypothesis": ..., "model": "random_forest" | "gradient_boosting", "params": {...}}
{"action": "interpret", "interpretation": ..., "decision": "continue" | "stop", "final_recommendation": ...}
```

Everything outside the five marked slots is byte identical across all 32 files of a task. The three remaining substitutions are set by the dataset, so they are constant within a task and differ only between tasks.

The ten paragraphs, verbatim (1 of 2)

slot M Evaluation criterion

- [0] Evaluate model quality in whatever way you judge appropriate.
- [1] Evaluate every model using 5-fold cross-validated accuracy on the training split, AND report the standard deviation across folds as a measure of stability. A configuration is only better than another if its mean accuracy is higher; use the standard deviation only to break ties or to flag instability.

slot B Search breadth

- [0] Stay within a single model family per experiment. Pick either RandomForestClassifier or GradientBoostingClassifier for the whole session unless you have exhausted reasonable hyperparameters for it and explicitly decide to switch.
- [1] You are free to compare RandomForestClassifier and GradientBoostingClassifier side by side within the same session, and to vary multiple hyperparameters of each at once.

slot S Stopping rule

- [0] You have a fixed budget of exactly 3 experiments in total. Run exactly 3, then stop and report your final recommendation, regardless of the outcome of experiment 3.
- [1] You have a budget of up to 8 experiments. After each experiment, decide whether to continue: stop as soon as an experiment fails to improve the evaluation metric (defined above) by more than 0.002 over the best result so far, or when you reach 8 experiments, whichever comes first.

The ten paragraphs, verbatim (2 of 2)

slot 0 Reporting style

- [0] Keep every response short: state the action you are taking and the numbers involved, with minimal prose.
- [1] Explain your reasoning at every step in full sentences: what you hypothesize, why, what you expect to observe, and how you interpret the actual result relative to that expectation.

slot E Search emphasis

- [0] Prioritize breadth of search: try several meaningfully different configurations before you spend more than one experiment refining any single one of them.
- [1] Prioritize depth: as soon as one configuration looks promising, immediately refine it with nearby variations (e.g. slightly different hyperparameter values) before trying anything unrelated.

Why slot O is a lever, not a style preference

The loop carries no state but its own transcript. What the model wrote in the hypothesis and interpretation fields of the previous turns is the only thing it can read back before proposing the next experiment. Slot O governs how much of the loop's reasoning is written down and available to itself.

What a configuration is, concretely

$c = (1, 0, 1, 1, 0)$ is the file M1-B0-S1-01-E0: paragraph [1] in the evaluation slot, [0] in breadth, [1] in stopping, [1] in reporting, [0] in emphasis. Its neighbour M1-B0-S1-00-E0 is the same file with one paragraph swapped and nothing else changed. The other 30 files are the remaining combinations.

The complete factorial makes interactions estimable

- With all 32 files, every pairwise interaction β_{ij} is estimated separately

This is what makes RQ2 testable

Does the effect of one component depend on the level of another?

Tasks, executor and controls

- Breast Cancer Wisconsin, 569×30 , two classes
- Wine, 178×13 , three classes
- One stratified 70/30 split per task
- Two model families under a fixed hyperparameter whitelist: the set of hyperparameters the agent is allowed to name for each family, anything outside it being rejected before an estimator is built

Held fixed, so they explain nothing

agent model gpt-4o-mini-2024-07-18, temperature = 0.7. Runs are separate conversations.

The baseline $a_0(d)$: exactly two fits per dataset

One `RandomForestClassifier` and one `GradientBoostingClassifier`. So with scikit-learn's own defaults for training. $a_0(d)$ is the better of the two; on both datasets it is the random forest: 0.9598 on Breast Cancer and 0.9600 on Wine.

Design and budget

$$N = \underbrace{2^5}_{\text{configurations}} \times \underbrace{D}_{\text{2 tasks}} \times \underbrace{R}_{\text{20 replicates}} = 1,280 \text{ runs, 64 cells, 20 each}$$

- Of the 1,280 runs, 1,278 terminated on the model's own stop decision and 2 on the cap.

The dependent variables: what the run achieved

Gain over default

The difference between the score of the configuration the run finally recommends and the baseline $a_0(d)$. It answers: did the tuning achieve anything that untuned defaults would not have given. A negative value is a real failure; a run that got worse than the one it started from.

Improvement rate

The proportion of executed trials whose score is strictly higher than the best score seen earlier in the same run. A high rate is evidence that the interpret step is genuinely informing the next proposal.

The dependent variables: what the search cost

Wasted trial ratio

The proposals rejected before execution. Malformed or outside the whitelist or exactly duplicate an earlier proposal in the same run, divided by all proposals attempted. A high ratio means the loop is either failing to satisfy the response format or failing to consult its own transcript.

Cost to best

The number of executor calls up to and including the one that produced the run's highest score.

The first two are outcome metrics, the second two efficiency metrics, and the results turn on that distinction. All four are read straight off the structured transcript: each is a difference in accuracy, a proportion of proposals, or a count of executor calls, so no judge model or scoring library enters the measurement and nothing in it can drift with the instruction being tested.

The measurement window

Resolution

All four metrics are computed over the **first three experiments**, that being the number performed at the fixed budget level and hence the number available in every run.

The regression model

Orthogonal coding, then main effects with the task controlled

$$X_i = +1 \text{ at level 1, } X_i = -1 \text{ at level 0}$$

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{j=2}^D \gamma_j D_j + \varepsilon$$

Interactions, free because all 32 cells were run

$$Y = \beta_0 + \sum_i \beta_i X_i + \sum_{i < j} \beta_{ij} X_i X_j + \sum_j \gamma_j D_j + \varepsilon$$

- $2\beta_i$ is the average change in Y from switching component i , and is what is reported throughout

What the 1,280 runs actually did

Per run	mean	median	max
Proposals attempted	4.02	3	9
Experiments executed	3.73	3	8
Proposals rejected	0.29	0	2
Duplicate proposals	0.004	0	1
Executor calls to best	1.71	1	7

Study totals

5,140 proposals attempted, 4,768 experiments executed, 372 rejected, 5 duplicates.

How runs ended

1,278 on the model's own stop decision, 2 on the 8 experiment cap, none on the 4 bad reply cap.

Per run	$S = 0$ fixed 3	$S = 1$ adaptive	$O = 0$ terse	$O = 1$ full sentences
Experiments executed	2.88	4.57	3.76	3.69
Proposals rejected	0.17	0.41	0.43	0.15
Experiments after best	1.30	2.73	2.05	1.99

When the best score arrived, and where the waste came from

Best on call	runs	share
1	720	56.2%
2	291	22.7%
3	222	17.3%
4	27	2.1%
5	14	1.1%
6	5	0.4%
7	1	0.1%

Most runs peak almost at once

56% of runs never improve on their first trial and 96.2% never improve after their third. On average 2.02 experiments per run are executed after the best has already been found: 2,585 of the 4,768 trials, more than half the compute in the study.

Where the waste came from

All 372 rejections were whitelist violations, 365 of them on the random forest. Not one reply was malformed JSON or the wrong shape, and only 5 proposals in the whole study duplicated an earlier one.

Main effect estimates $2\hat{\beta}_i$

Metric	M	B	S	O	E	$\hat{\gamma}$	R^2
Gain over default	+0.001	-0.001	+0.001**	+0.000	+0.000	-0.001***	0.029
Wasted trial ratio	-0.023***	-0.034***	+0.044***	-0.075***	+0.005	-0.003	0.159
Improvement rate	+0.008	-0.026*	+0.034**	+0.006	+0.018	-0.064***	0.044
Cost to best	+0.06	-0.02	+0.05	-0.01	-0.11	-0.20***	0.025

- Reporting style is the largest effect anywhere in the design, -0.075 , cutting the wasted trial ratio from 0.106 to 0.031
- The adaptive stopping rule works the other way, $+0.044$: it raises the mean number of experiments from 2.88 to 4.57, and some of the extra proposals duplicate earlier ones
- The stopping rule is the only component touching either outcome metric, and nothing survives on cost to best
- On two metrics the dataset effect $\hat{\gamma}$ is about twice the largest component effect, both at $p < 10^{-5}$. That is exactly the variance γ_j takes out of the residual

HC3 errors, Benjamini and Hochberg adjusted p . $\hat{\gamma}$ is Wine relative to Breast Cancer. Stars: adjusted $p < 0.05$, < 0.01 , < 0.001 .

Limitations

- ① **One agent model.** the actual implication in Claude does not tell us anything about the number of agents spawned
- ② **One wording per level.** phrasing was not varied for robustness
- ③ **Two datasets.**

Conclusion

- `program.md` does what controller code does in conventional software, yet cannot be read the way code is read, so it was treated as a configuration vector
- All $2^5 = 32$ files, two datasets, twenty replicates, 1,280 runs against a real executor
- Because the configurations were constructed rather than collected, the coefficients admit a causal reading. Because the factorial is complete, independence could be tested rather than assumed
- **RQ1:** the components determine the process, not the outcome, and reporting style dominates
- **RQ2:** three of forty interactions survive, all involving reporting style, so the components are close to additive

For tasks of this difficulty, `program.md` determines how the loop conducts its search rather than what the search returns.